Secure QR-based Device Pairing and Encrypted Communication System

A PROJECT REPORT SUBMITTED IN FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING

By
Batch – B12

K.Bindu (323103210104)
K.Likitha (323103210245)
D.Yamuna(323103210240)
K.Revathi (323103210246)

Under the esteemed guidance of
Dr. N. Sharmili
Professor
CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]
[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]
[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]
Kommadi, Madhurawada, Visakhapatnam – 530048

2023 – 2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project report titled "**Secure QR-based Device Pairing and Encrypted Communication System"** is a bonafide work of following III B.Tech. –II Sem students in the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women affiliated to Andhra University, Visakhapatnam during the academic year 2025-26, in fulfilment of the requirement for the award of the degree of Bachelor of Technology of this university.

K.Bindu (323103210104)
D.Yamuna (323103210240)
K.Likitha (323103210245)
K.Revathi (323103210246)

Signature of the Guide

Dr. N. Sharmili MTech., Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

INDEX:

1.	SYSTEM REQUIREMENTS 1.1. Software Requirements 1.2 Hardware Requirements	1-2
2.	METHODOLOGY 2.1 System Architecture 2.2 Description, Algorithm	3-5
3.	IMPLEMENTATION 3.1Source code	6-17
4.	RESULTS 4.1 Output Screens	18-23
5.	CONCLUSION	24
6.	FUTURE SCOPE	25
7.	REFERENCES	26

ABSTRACT

This project presents a secure device pairing and communication system using QR codes and cryptography techniques. In today's digital world, secure data transfer between devices is very important to protect sensitive information from unauthorized access. The system uses a QR code-based method to establish a connection between two devices quickly and securely.

The project implements the Diffie-Hellman key exchange algorithm to generate a shared secret key between the devices. This key is then used for encrypting messages and files using symmetric encryption techniques. The system ensures that all communication between devices remains confidential and secure.

The application is developed using Python and Flask, providing a user-friendly interface for communication. The project demonstrates how cryptographic techniques can be effectively used to build a secure and reliable communication system

SYSTEM REQUIREMENTS

1.1. Software Requirements

Table 1. Software Requirements

Software	Version / Specification
Python	3.x
Visual Studio Code Editor	latest
Flask	latest
Operating System	Windows/Linux/Mac
Cryptography Library	latest
QR Code Library	latest

Python packages

Python packages we used in our project includes the following table 2.

We used the command pip install "*package_name==version*" to download the packages.

Table 2. Python packages

S.No.	Package Used	Description
1.	NumPy	Used for numerical operations
2.	Pandas	Used for data handling (if required)
3.	Flask	Web framework for backend
4.	Cryptography	Used for encryption and decryption
5.	QR code	Used to generate QR codes
6.	JSON	Used for data exchange
7.	OS	Used for file handling
8.	Base64	Used for encoding/decoding data

Front End Packages used

Table 3. Front end Packages

S.NO	Libraries Used	Description
1.	HTML	Structure of web pages
2.	CSS	Styling of web pages
3.	Jinja2	Template engine used in Flask
4.	JavaScript	For basic interactivity

1.2 Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

Minimum Hardware Requirements:

Table 4. Minimum Hardware Requirements

S.No	Hardware Component	Specification
1.	Processor	Intel i3 or above
2.	RAM	4 GB or above
3.	Hard Disk	500 GB
4.	System Type	64-bit System
5.	Input Devices	Keyboard, Mouse
6.	Output Devices	Monitor

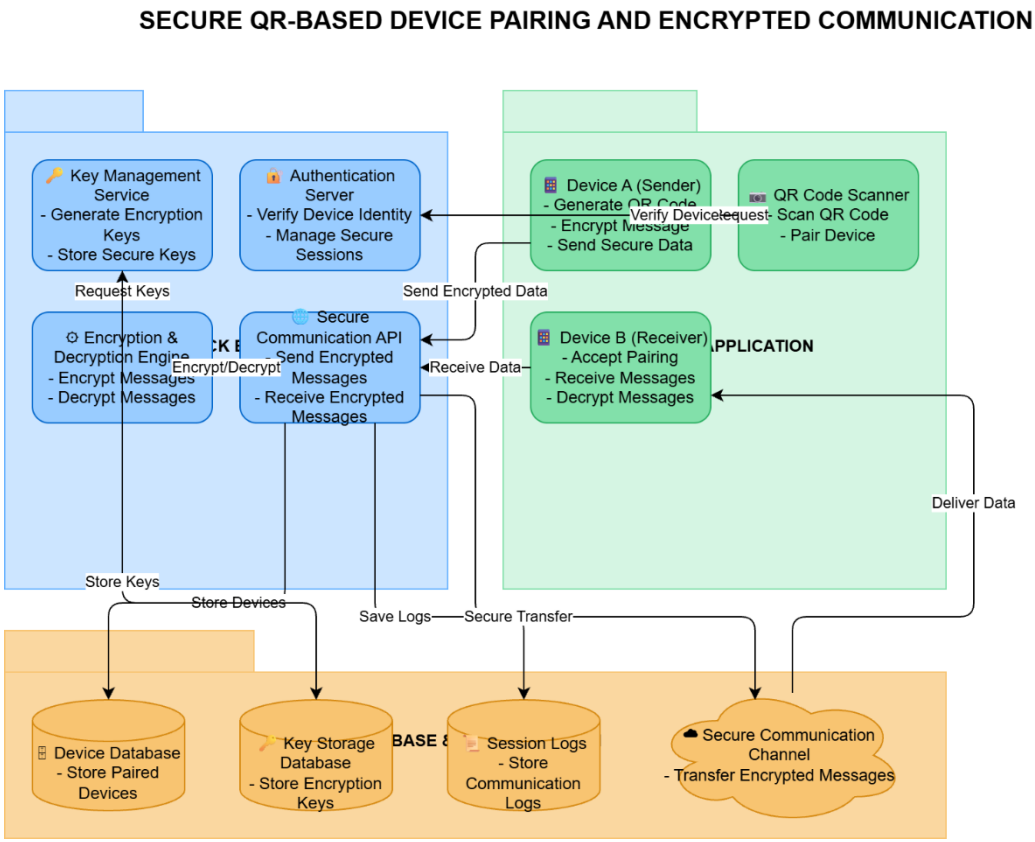
2.METHODOLOGY

2.1 SYSTEM ARCHITECTURE

The system consists of two main devices, namely Device A and Device B, which communicate securely through a web-based interface. The communication process begins when Device A generates a session and a QR code. This QR code contains the necessary information required for pairing.

Device B scans the QR code to join the session. After successful pairing, both devices perform the Diffie-Hellman key exchange algorithm to generate a shared secret key. This key is not transmitted directly but is computed independently by both devices, ensuring security.

Once the shared key is generated, it is used for encrypting and decrypting messages and files using symmetric encryption techniques. The system ensures that all communication between devices is secure and protected from unauthorized access.

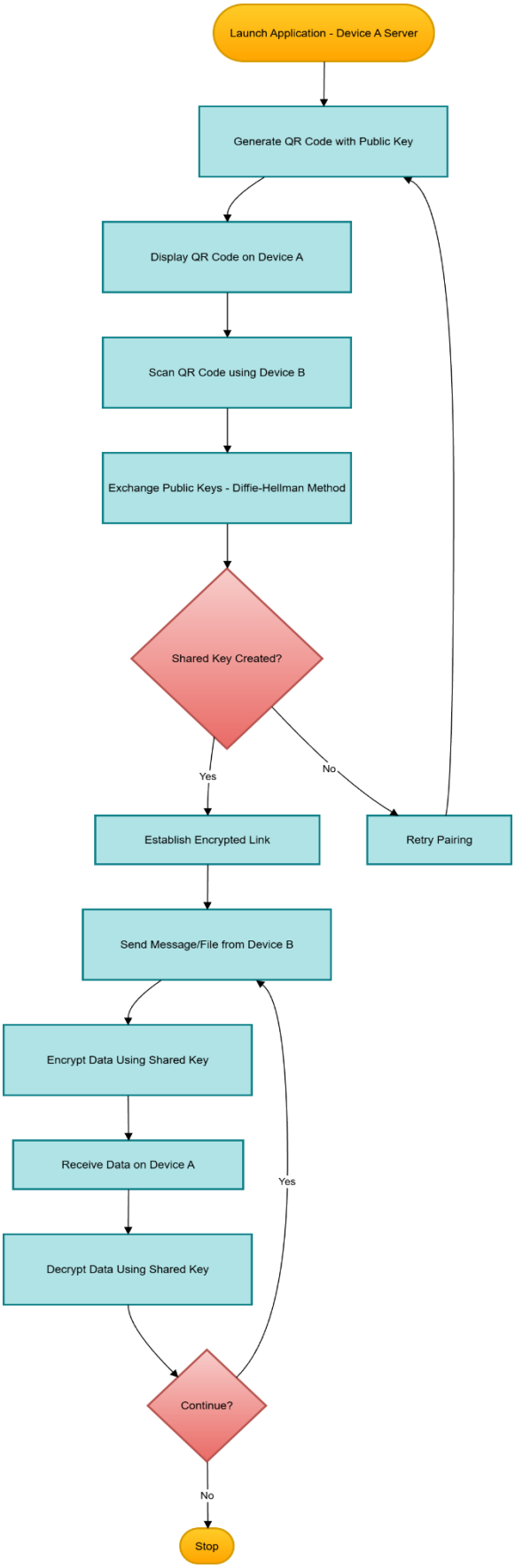


SYSTEM ARCHITECTURE

2.2DESCRPTION/ algorithm

Algorithm: Secure Device Pairing and Communication

1. Start the application using Flask server.
2. Device A creates a session and generates a QR code.
3. Device B scans the QR code to join the session.
4. Both devices initiate Diffie-Hellman key exchange.
5. Public keys are exchanged between devices.
6. Each device computes the shared secret key independently.
7. The shared key is used to generate a symmetric encryption key.
8. Messages and files are encrypted using the encryption key.
9. Encrypted data is transmitted between devices.
10. Receiver decrypts the data using the same key.
11. Secure communication continues until session ends.


Launch Application - Device A Server
Generate QR Code with Public Key
Display QR Code on Device A
Scan QR Code using Device B
Exchange Public Keys - Diffie-Hellman Method
Shared Key Created?
Yes
No
Establish Encrypted Link
Retry Pairing
Send Message/File from Device B
Encrypt Data Using Shared Key
Receive Data on Device A
Yes
Decrypt Data Using Shared Key
Continue?
No
Stop

FLOW CHART

3. IMPLEMENTATION

The project is implemented using Python and Flask framework to create a web-based secure communication system. The application consists of multiple modules that handle device pairing, key exchange, encryption, and data transmission.

Initially, the Flask server is started, which manages the communication between devices. Device A generates a session and creates a QR code containing session details. Device B scans this QR code to join the session, enabling both devices to connect.

After pairing, the Diffie-Hellman key exchange algorithm is used to generate a shared secret key. This key is not transmitted over the network but is independently calculated by both devices, ensuring secure key exchange.

The shared key is then used with the Fernet encryption method to encrypt messages and files before transmission. When a message or file is sent, it is first encrypted and then transmitted over the network. The receiving device decrypts the data using the same key, ensuring confidentiality and security.

The system also supports file transfer by encrypting the file data before sending and decrypting it after receiving. All operations are handled through a user-friendly web interface.

Source code :

app.py:

```
from flask import Flask, render_template, request, jsonify, send_file

from crypto_utils import *

import uuid, qrcode, io, json


app = Flask(__name__)


sessions = {}

messages = {}

files = {}
```

```
@app.route('/')
def home():
    return "<h2>Go to /deviceA or /deviceB</h2>"

@app.route('/deviceA')
def deviceA():
    return render_template('deviceA.html')

@app.route('/deviceB')
def deviceB():
    return render_template('deviceB.html')

# Create QR
@app.route('/create_qr')
def create_qr():
    session_id = str(uuid.uuid4())
    privateA, publicA = generate_keys()

    sessions[session_id] = {
        "A_private": privateA,
        "A_public": publicA,
        "shared_key": None
    }

    data = {
        "session_id": session_id,
        "public_key": serialize_public_key(publicA).decode()
    }
```

```
    qr = qrcode.make(json.dumps(data))
    img = io.BytesIO()
    qr.save(img, 'PNG')
    img.seek(0)

    return send_file(img, mimetype='image/png')

# Send session_id to Device A
@app.route('/get_session')
def get_session():
    session_id = list(sessions.keys())[-1]
    return jsonify({"session_id": session_id})

# Scan QR
@app.route('/scan_qr', methods=['POST'])
def scan_qr():
    qr_data = json.loads(request.json['qr_data'])

    session_id = qr_data['session_id']
    pubA = load_public_key(qr_data['public_key'].encode())

    privateB, publicB = generate_keys()

    sharedA = generate_shared_key(
        sessions[session_id]["A_private"], publicB
    )
    sharedB = generate_shared_key(privateB, pubA)

    if sharedA == sharedB:
```

```
        sessions[session_id]["shared_key"] = sharedA
        return jsonify({"status": "Paired", "session_id": session_id})
    return jsonify({"status": "Failed"})

# Send message
@app.route('/send', methods=['POST'])
def send():
    data = request.json
    session_id = data['session_id']
    key = sessions[session_id]["shared_key"]

    encrypted = encrypt_message(key, data['message'].encode())
    messages[session_id] = encrypted

    return jsonify({"status": "sent"})

# Get message
@app.route('/get_message/<session_id>')
def get_message(session_id):
    if session_id in messages:
        return jsonify({"msg": messages.pop(session_id)})
    return jsonify({"msg": None})

# Decrypt message
@app.route('/receive', methods=['POST'])
def receive():
    data = request.json
    key = sessions[data['session_id']]["shared_key"]
```

```
    decrypted = decrypt_message(key, data['encrypted']).decode()
    return jsonify({"decrypted": decrypted})

# Upload file
@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    session_id = request.form['session_id']

    key = sessions[session_id]["shared_key"]
    encrypted = encrypt_message(key, file.read())

    files[session_id] = {
        "filename": file.filename,
        "data": encrypted
    }

    return jsonify({"status": "uploaded"})

# Get file
@app.route('/get_file/<session_id>')
def get_file(session_id):
    if session_id in files:
        return jsonify(files.pop(session_id))
    return jsonify({"filename": None})

# Decrypt file
@app.route('/download', methods=['POST'])
def download():
```

```
    data = request.json
    key = sessions[data['session_id']]["shared_key"]

    decrypted = decrypt_message(key, data['data'])

    return jsonify({
        "filename": data['filename'],
        "content": decrypted.decode('latin1')
    })

if __name__ == '__main__':
app.run(debug=True, host='0.0.0.0', threaded=True,port=5000, ssl_context='adhoc')
```

cypto_utils:

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.fernet import Fernet
import base64

parameters = dh.generate_parameters(generator=2, key_size=2048)

def generate_keys():
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key

def serialize_public_key(public_key):
```

```
    return public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )

def load_public_key(pem):
    return serialization.load_pem_public_key(pem)

def generate_shared_key(private_key, peer_public_key):
    shared = private_key.exchange(peer_public_key)

    derived = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake'
    ).derive(shared)

    return base64.urlsafe_b64encode(derived)

def encrypt_message(key, message):
    return Fernet(key).encrypt(message).decode()

def decrypt_message(key, cipher):
return Fernet(key).decrypt(cipher.encode())
```

deviceA:

```
<!DOCTYPE html>
```

```
<html>
<body align="center">

<h2>Device A</h2>

<button onclick="genQR()">Generate QR</button>
<br><br>

<img id="qr" width="250">

<h3>Received Message:</h3>
<p id="msg"></p>

<h3>Received File:</h3>
<a id="file"></a>

<script>
let session_id = "";

async function genQR() {
    let res = await fetch('/create_qr');
    let blob = await res.blob();
    document.getElementById("qr").src = URL.createObjectURL(blob);

    let s = await fetch('/get_session');
    let data = await s.json();
    session_id = data.session_id;
}
```

```
// Poll
setInterval(async () => {
    if (!session_id) return;

    let res = await fetch(`/get_message/${session_id}`);
    let data = await res.json();

    if (data.msg) {
        let dec = await fetch('/receive', {
            method:'POST',
            headers:{'Content-Type':'application/json'},
            body: JSON.stringify({session_id, encrypted:data.msg})
        });
        let d = await dec.json();
        document.getElementById("msg").innerText = d.decrypted;
    }

    let f = await fetch(`/get_file/${session_id}`);
    let file = await f.json();

    if (file.filename) {
        let down = await fetch('/download', {
            method:'POST',
            headers:{'Content-Type':'application/json'},
            body: JSON.stringify({session_id, ...file})
        });

        let d = await down.json();
        let blob = new Blob([d.content]);
```

```
            let url = URL.createObjectURL(blob);

            let link = document.getElementById("file");
            link.href = url;
            link.download = d.filename;
            link.innerText = "Download " + d.filename;
        }

    }, 5000);
    </script>

    </body>
</html>
```

deviceB:

```
    <!DOCTYPE html>
    <html>
    <head>
    <script src="https://unpkg.com/html5-qrcode"></script>
    </head>
    <body align="center">

    <h2>Device B</h2>

    <div id="reader" style="width:300px;"></div>
    <p id="status"></p>

    <input id="msg" placeholder="Message">
```

```
<button onclick="sendMsg()">Send</button>

<h3>File</h3>
<input type="file" id="fileInput">
<button onclick="uploadFile()">Send File</button>

<script>
let session_id = "";

function onScanSuccess(text) {
    fetch('/scan_qr', {
        method:'POST',
        headers:{'Content-Type':'application/json'},
        body: JSON.stringify({qr_data:text})
    })
    .then(res=>res.json())
    .then(data=>{
        session_id = data.session_id;
        document.getElementById("status").innerText = data.status;
    });
}

new Html5QrcodeScanner("reader",{fps:10}).render(onScanSuccess);

async function sendMsg() {
    let msg = document.getElementById("msg").value;

    await fetch('/send', {
        method:'POST',
```

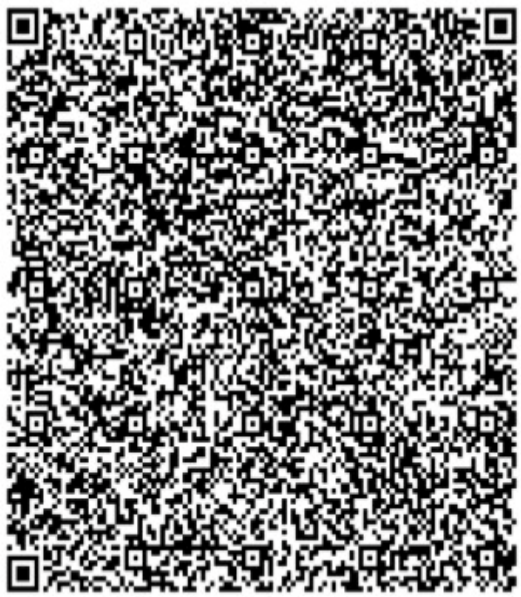
```
            headers:{'Content-Type':'application/json'},
            body: JSON.stringify({session_id,message:msg})
        });
    }

    async function uploadFile() {
        let file = document.getElementById("fileInput").files[0];

        let form = new FormData();
        form.append("file", file);
        form.append("session_id", session_id);

        await fetch('/upload',{method:'POST',body:form});
    }
    </script>

    </body>
</html>
```

4. RESULTS

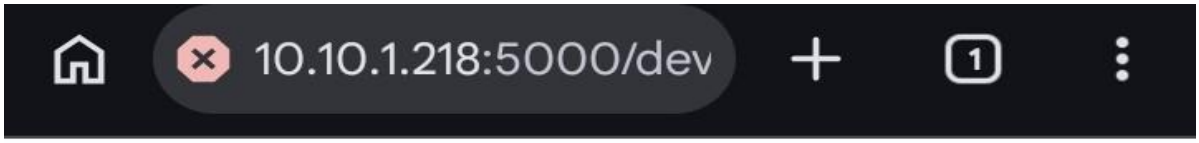
4.1 Output Screens

4.1.1

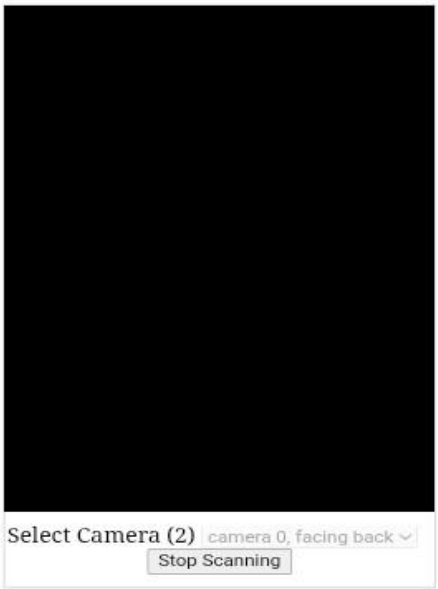
4.1.2



10.10.1.218:5000/dev
Device B
Request Camera Permissions
Scan an Image File
Message
Send
File
Choose file
No file chosen
Send File



Device B



Paired

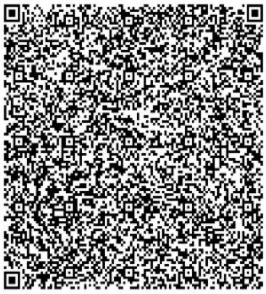
Hello Hii Send

File

Choose file No file chosen Send File

Device A

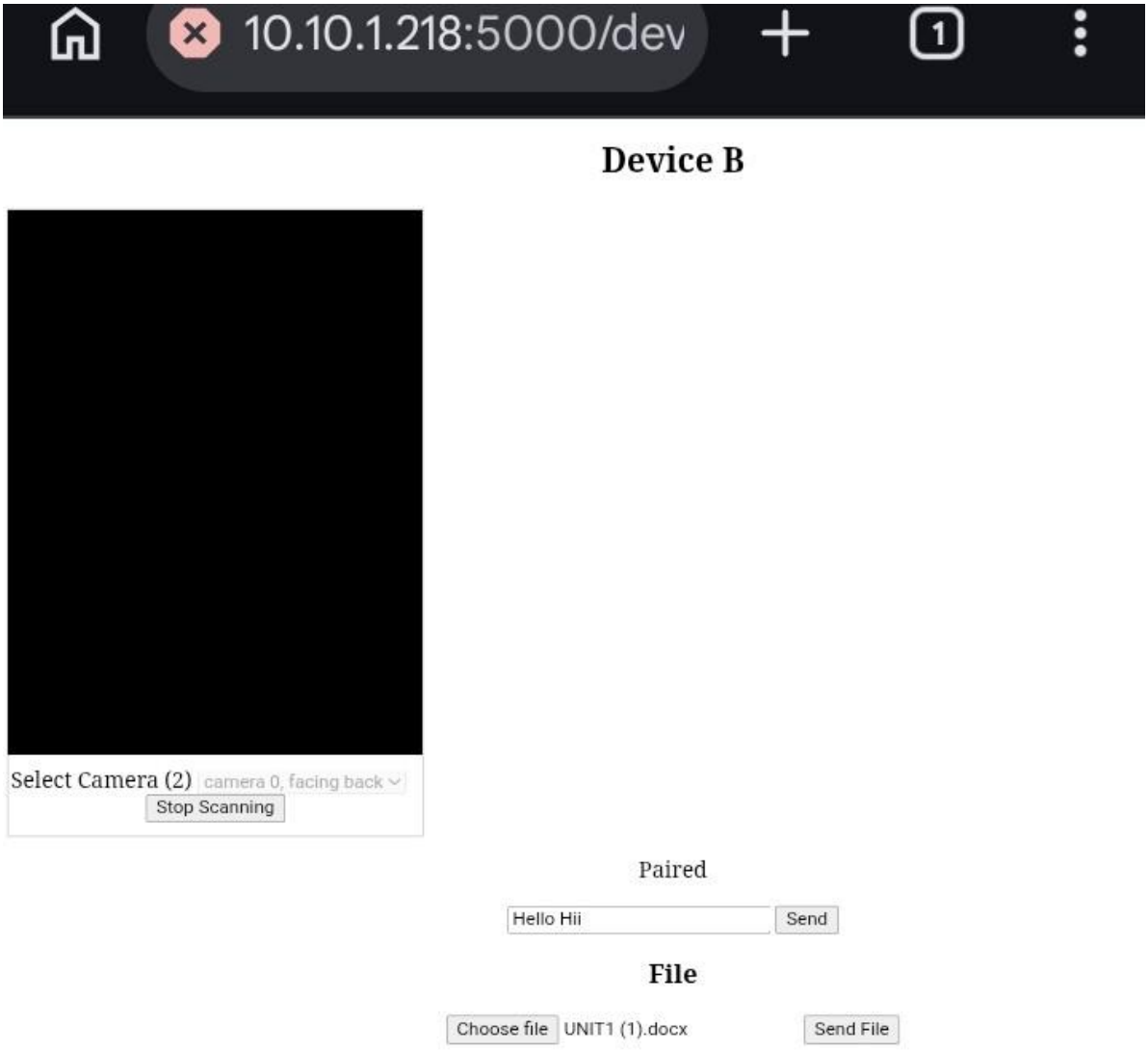
Generate QR

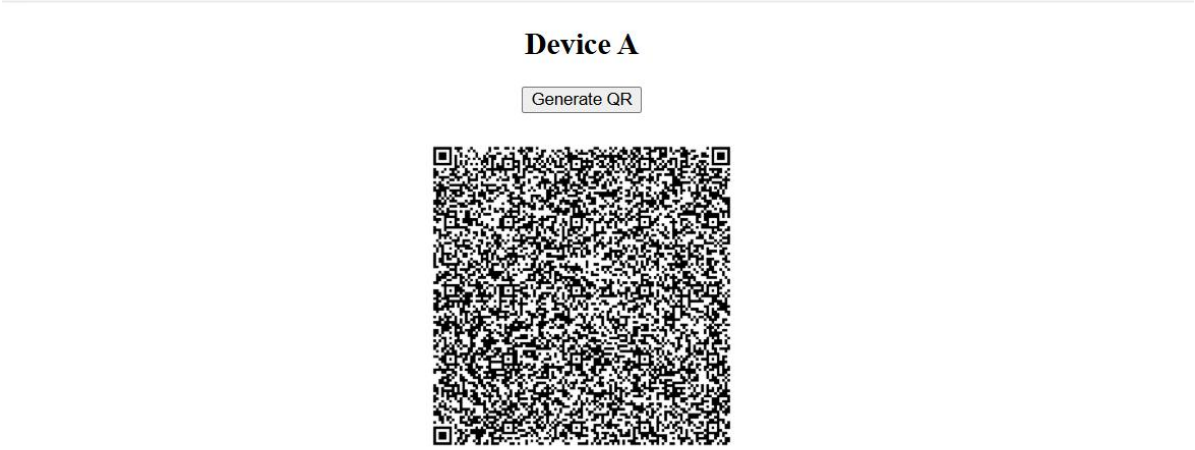


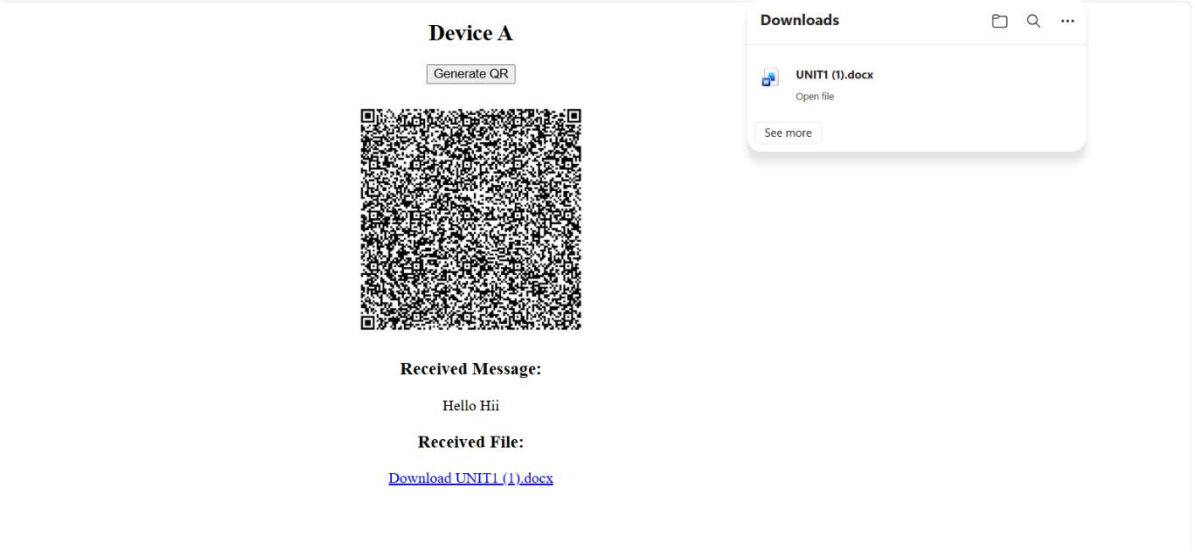
Received Message:

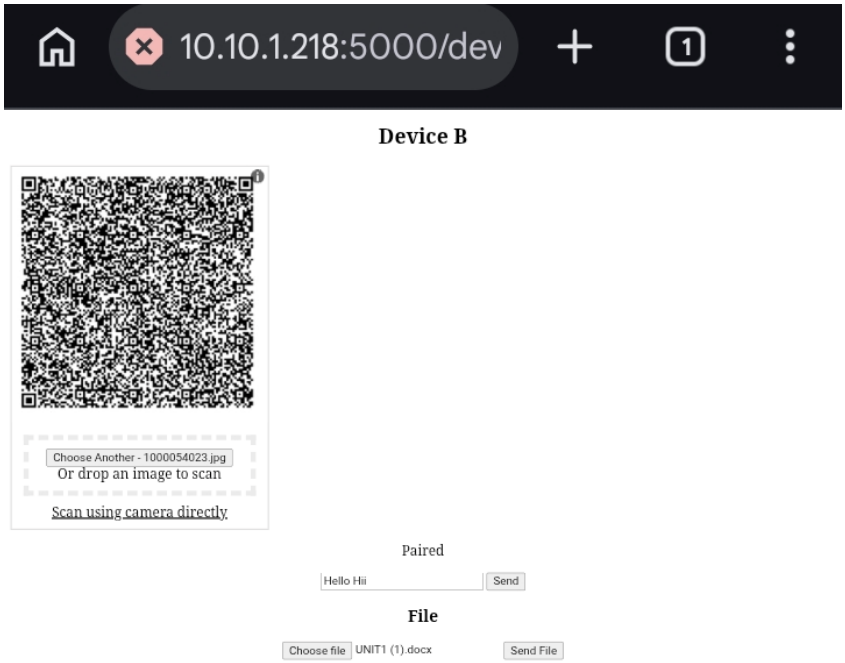
Hello Hii

Received File:


10.10.1.218:5000/dev
Device B
Select Camera (2)
camera 0, facing back
Stop Scanning
Paired
Hello Hii
Send
File
Choose file
UNIT1 (1).docx
Send File


Device A
Generate QR
Received Message:
Hello Hii
Received File:
Download UNIT1 (1).docx


Device A
Generate QR
Received Message:
Hello Hii
Received File:
Download UNIT1 (1).docx
Downloads
UNIT1 (1).docx
Open file
See more


10.10.1.218:5000/dev
Device B
Choose Another - 1000054023.jpg
Or drop an image to scan
Scan using camera directly
Paired
Hello Hii
Send
File
Choose file
UNIT1 (1).docx
Send File

5.CONCLUSION

The project successfully demonstrates a secure device pairing and communication system using QR codes and cryptographic techniques. It provides a simple and efficient way for two devices to establish a secure connection without manual configuration.

By using the Diffie-Hellman key exchange algorithm, the system ensures that a shared secret key is generated securely without transmitting it over the network. The use of encryption techniques helps in protecting messages and files from unauthorized access.

The implementation using Python and Flask makes the system user-friendly and easy to operate. Overall, the project highlights the importance of secure communication and shows how modern cryptographic methods can be applied in real-world applications.

6.FUTURE SCOPE

The project can be further enhanced by adding advanced security features and improving usability. In the future, multi-device communication can be implemented, allowing more than two devices to connect and communicate securely at the same time.

Additional authentication mechanisms such as biometric verification or OTP-based login can be integrated to increase security. The system can also be extended to support real-time communication features like voice and video calls.

Moreover, the application can be developed as a mobile app to make it more accessible and user-friendly. Cloud integration can be added to enable secure storage and sharing of files across different platforms.

Overall, the project has strong potential for expansion and can be developed into a complete secure communication platform.

7.REFERENCES

· W. Diffie and M. Hellman, “New Directions in Cryptography,” IEEE Transactions on Information Theory, 1976.
· Official Python Documentation – https://www.python.org
· Flask Documentation – https://flask.palletsprojects.com
· Cryptography Library Documentation – https://cryptography.io
· QR Code Generation Library – https://pypi.org/project/qrcode/
· William Stallings, “Cryptography and Network Security: Principles and Practice,” Pearson Education.